

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
федеральное государственное бюджетное образовательное учреждение
высшего образования
**«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»**

Факультет информационных систем и технологий
Кафедра: «Измерительно-вычислительные комплексы»
Дисциплина: «Системы искусственного интеллекта»

Отчёт
По лабораторной работе №2

Выполнила:
студент гр. ИСТбд-32
Минибаева Е.Р.
Проверил:
к. т. н., доцент каф. ИВК
Шишкин В.В.

Ульяновск
2026 г.

Введение

Целью лабораторной работы являлось практическое освоение методов кластеризации на примере задачи группировки городов по географическим координатам. Кластеризация относится к задачам обучения без учителя: необходимо разбить набор точек (городов) на K компактных групп так, чтобы объекты внутри одной группы были максимально близки друг к другу, а группы — максимально удалены. В работе требовалось реализовать два классических алгоритма кластеризации — агломеративную иерархическую кластеризацию (с критерием single-link) и алгоритм K -means — как вручную, так и с использованием готовой реализации из библиотеки `scikit-learn`, а затем сравнить их по качеству и быстродействию на нескольких синтетических и реальных конфигурациях данных. В отчете приведено описание использованных методов, тестовых наборов, результаты экспериментов и их анализ.

Используемые методы

Агломеративная иерархическая кластеризация

Агломеративный подход строит иерархию кластеров снизу вверх. Изначально каждая точка считается отдельным кластером. На каждом шаге два ближайших (по заданной метрике расстояния между кластерами) кластера объединяются в один. Процесс повторяется до тех пор, пока не останется ровно K кластеров. В данной работе использовалось single-link расстояние между кластерами, то есть минимальное расстояние между любой точкой одного кластера и любой точкой другого. Евклидово расстояние (вычисляемое через гипотенузу разности широт и долгот) служило мерой близости между отдельными городами. Этот метод хорошо выявляет цепочечные структуры, но может быть чувствителен к шуму. Реализация алгоритма выполнена вручную: поддерживается список списков индексов кластеров, для каждого шага перебираются все пары кластеров, вычисляется минимальное попарное расстояние, и два самых близких объединяются.

Алгоритм K -means (ручная реализация)

Алгоритм K-means относится к центроидным методам. Он требует априорного задания числа кластеров K . Процедура состоит из следующих шагов: случайная инициализация K центроидов (координат центров кластеров) в пределах минимальных и максимальных значений широт и долгот; затем на каждой итерации каждая точка приписывается к ближайшему центроиду (по евклидову расстоянию), после чего центроиды пересчитываются как средние арифметические координат точек, попавших в соответствующий кластер. Алгоритм повторяется до сходимости (центроиды перестают изменяться) или до достижения максимального числа итераций. В ручной реализации учтена возможность появления пустых кластеров — для них новый центроид выбирается случайно. Этот алгоритм стремится минимизировать сумму квадратов расстояний от точек до центроидов (инерцию), но может сходиться к локальному, а не глобальному минимуму.

K-means из библиотеки scikit-learn

Для сравнения использовался стандартный класс KMeans из модуля `sklearn.cluster`. Он предоставляет оптимизированную реализацию алгоритма с улучшенной инициализацией (по умолчанию «k-means++»), множественными запусками (`n_init=10`) и эффективными вычислениями. В эксперименте параметры задавались так: `n_clusters=K`, `random_state=42` для воспроизводимости, `n_init=10`. Этот алгоритм служит эталоном качества и производительности.

Метрики оценки качества кластеризации

Для объективного сравнения алгоритмов использовались три метрики.

Инерция (`inertia`) — сумма квадратов евклидовых расстояний от каждой точки до центроида её кластера. Чем меньше инерция, тем компактнее кластеры, но при увеличении K инерция всегда уменьшается, поэтому сравнивать разные K напрямую нельзя. В данном эксперименте K фиксировано для каждой конфигурации, поэтому инерция служит корректной мерой.

Индекс силуэта (silhouette score) вычисляется для каждой точки как разность между средним расстоянием до точек своего кластера и средним расстоянием до точек ближайшего соседнего кластера, нормированная на максимальное из этих двух значений. Итоговый силуэт — среднее по всем точкам. Он лежит в диапазоне от -1 до 1 , где 1 означает идеально разделённые компактные кластеры, 0 — пересекающиеся или нечёткие кластеры, а отрицательные значения — ошибочную кластеризацию. Индекс силуэта не требует знания истинных меток и применим для оценки относительного качества разбиения.

Время выполнения замерялось с помощью `time.perf_counter()` для каждого алгоритма на каждой конфигурации, что позволило сравнить их вычислительную эффективность.

Тестовые конфигурации

Для проверки алгоритмов в различных условиях были подготовлены пять наборов данных, моделирующих разные географические ситуации.

Три компактные группы — девять точек, образующих три явно разделённых облака (по три точки в каждом). Этот тест проверяет способность алгоритмов правильно выделять естественные кластеры при отсутствии шума и наложений.

Случайные точки — пятнадцать точек со случайными координатами в диапазоне от 0 до 100 . Здесь нет явной кластерной структуры; алгоритмы могут давать разные разбиения, и качество оценивается по силуэту и инерции.

Линейная структура — двадцать точек, лежащих почти на прямой линии (зависимость $\text{lon} = 2 * \text{lat}$). Такая структура является сложной для K-means, так как алгоритм предполагает сферические кластеры, а иерархическая кластеризация с single-link, наоборот, хорошо выделяет вытянутые цепочки.

Один выброс — пять точек, сгруппированных вблизи (10,10...14,14), и одна удалённая точка (100,100). Проверяется устойчивость алгоритмов к аномальным значениям.

Исходные города — пять городов с произвольными координатами, взятыми из условия задачи. Требуется разбить их на два кластера. Это имитация реального, но небольшого набора данных.

Результаты экспериментов

Вывод программы:

```
=== 3 компактные группы (K=3) ===
```

```
Агломеративный: inertia=6.00, silhouette=0.953, time=0.00017s
```

```
K-means ручной: inertia=6.00, silhouette=0.953, time=0.00041s
```

```
K-means sklearn: inertia=6.00, silhouette=0.953, time=1.57043s
```

```
=== Случайные точки (K=3) ===
```

```
Агломеративный: inertia=9435.39, silhouette=0.405, time=0.00031s
```

```
K-means ручной: inertia=9435.39, silhouette=0.405, time=0.00036s
```

```
K-means sklearn: inertia=7332.99, silhouette=0.484, time=0.03232s
```

```
=== Линейная структура (K=2) ===
```

```
Агломеративный: inertia=2850.00, silhouette=0.178, time=0.00089s
```

```
K-means ручной: inertia=825.00, silhouette=0.590, time=0.00090s
```

```
K-means sklearn: inertia=825.00, silhouette=0.590, time=0.03216s
```

```
=== Один выброс (K=2) ===
```

```
Агломеративный: inertia=20.00, silhouette=0.814, time=0.00019s
```

```
K-means ручной: inertia=20.00, silhouette=0.814, time=0.00053s
```

```
K-means sklearn: inertia=20.00, silhouette=0.814, time=0.02790s
```

```
=== Исходные города (K=2) ===
```

```
Агломеративный: inertia=4005.80, silhouette=0.475, time=0.00015s
```

```
K-means ручной: inertia=4005.80, silhouette=0.475, time=0.00021s
```

```
K-means sklearn: inertia=3923.28, silhouette=0.416, time=0.03397s
```

Конфигурация «Три компактные группы»

Все три алгоритма показали идеальные результаты: инерция равна 6.00, индекс силуэта — 0.953, что свидетельствует о почти полной делимости групп. Агломеративная кластеризация и ручной K-means сработали быстро (0.00017 и 0.00041 секунды), тогда как реализация sklearn затратила значительно больше времени — 1.57 секунды. Это связано с тем, что в sklearn используется многократная инициализация ($n_init=10$) и более сложные проверки сходимости; на малом числе точек (9) накладные расходы становятся доминирующими.

Конфигурация «Случайные точки»

Здесь результаты разошлись. Агломеративная кластеризация и ручной K-means дали инерцию 9435.39 и силуэт 0.405. Реализация sklearn нашла значительно лучшее решение с инерцией 7332.99 и силуэтом 0.484. Это классический пример того, что ручной K-means с простой случайной инициализацией может застрять в локальном минимуме, в то время как sklearn благодаря k-means++ и множественным запускам находит более качественное разбиение. Время выполнения: агломеративный 0.00031 с, ручной 0.00036 с, sklearn 0.032 с. Хотя sklearn медленнее ручных реализаций, разница уже не столь драматична, и выигрыш в качестве оправдывает дополнительные затраты.

Конфигурация «Линейная структура»

Для этой конфигурации ($K=2$) агломеративная кластеризация дала инерцию 2850.00 и очень низкий силуэт 0.178. Это объясняется тем, что single-link склонен объединять точки в длинные цепочки, и при $K=2$ он может выделить два кластера, один из которых содержит почти все точки, а другой — несколько выбросов на конце, что приводит к плохой компактности. Ручной K-means и sklearn показали одинаковый результат: инерция 825.00, силуэт 0.590. Оба алгоритма смогли разбить линейную последовательность на два более сбалансированных по размеру кластера, разделив её примерно пополам.

Это демонстрирует преимущество K-means перед иерархическим single-link на вытянутых структурах, если требуется примерно равное количество точек в кластерах. Время: ручной K-means 0.00090 с, sklearn 0.032 с.

Конфигурация «Один выброс»

Здесь все три алгоритма полностью совпали: инерция 20.00, силуэт 0.814. Один выброс (точка (100,100)) был правильно выделен в отдельный кластер, а пять компактных точек — во второй.

Это хороший признак устойчивости: даже при наличии аномалии кластеризация остаётся осмысленной. Время: ручные реализации ~0.0002–0.0005 с, sklearn ~0.028 с.

Конфигурация «Исходные города»

На этом наборе из пяти точек с разбросом координат, разбиваемых на два кластера, ручная иерархическая и ручной K-means дали инерцию 4005.80 и силуэт 0.475. Алгоритм sklearn получил чуть меньшую инерцию (3923.28), но при этом силуэт оказался ниже (0.416). Это интересный случай: меньшая инерция не всегда означает лучшее разделение; возможно, sklearn сгруппировал точки так, что один из кластеров стал более размытым, что снизило силуэт.

Ручные методы в данном контексте дали более сбалансированное разбиение. Время: ручные ~0.00015 –0.00021 с, sklearn ~0.034 с.

Анализ

По результатам пяти экспериментов можно сделать несколько выводов. Ручные реализации агломеративной кластеризации и K-means очень быстры на малых выборках (менее 0.001 с), но качество ручного K-means сильно зависит от инициализации — на случайных точках он уступил sklearn. Агломеративная кластеризация с single-link хороша для компактных групп, но не подходит для линейных структур, где предпочтительнее K-means. Реализация sklearn, хотя и медленнее (особенно на сверхмалых данных из-за накладных расходов), обеспечивает более стабильное и часто лучшее качество благодаря современным методам инициализации и многократным запускам. При этом на наборе «Исходные города» лучший силуэт показали ручные алгоритмы, что напоминает о том, что «лучший» алгоритм зависит от конкретного набора данных и выбранной метрики.

Визуализация полученных кластеров (например, построение точечных диаграмм на плоскости) не требовалась в коде, но в реальных географических задачах она помогла бы наглядно сравнить разбиения. Однако приведённые числовые метрики (инерция и силуэт) дают достаточно информации для объективной оценки.

Заключение

В ходе лабораторной работы были разработаны и протестированы три реализации алгоритмов кластеризации для группировки городов по координатам. Агломеративная иерархическая кластеризация (single-link) показала хорошие результаты на компактных группах, но оказалась непригодной для линейных структур. Ручная реализация K-мыс, несмотря на простоту, в ряде случаев (компактные группы, выброс, исходные города) работает не хуже библиотечной, однако страдает от случайности инициализации. Реализация из scikit-learn, хотя и требует больше времени (особенно на очень малых выборках), обеспечивает наилучшее качество в большинстве тестов за счёт продуманных эвристик. Полученные результаты демонстрируют важность выбора алгоритма в зависимости от структуры данных и компромисса между скоростью и качеством. Для

реальных географических задач с умеренным количеством городов рекомендуется использовать библиотечный K-means с настройкой параметров, а для анализа иерархических связей — агломеративную кластеризацию с другими метриками (например, полной связью).

Приложение № 1 - Листинг программы

```
import math
import random
import time
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Агломеративная кластеризация
def hierarchical_clustering(lat, lon, k):
    n = len(lat)
    clusters = [[i] for i in range(n)]
    # матрица расстояний
    dist_matrix = [[math.hypot(lat[i]-lat[j], lon[i]-lon[j]) for j in
range(n)] for i in range(n)]

    while len(clusters) > k:
        # поиск пары кластеров с минимальным single-link расстоянием
        min_dist = float('inf')
        merge_i, merge_j = -1, -1
        for i in range(len(clusters)):
            for j in range(i+1, len(clusters)):
                d = min(dist_matrix[a][b] for a in clusters[i] for b in
clusters[j])
                if d < min_dist:
                    min_dist = d
                    merge_i, merge_j = i, j
            clusters[merge_i].extend(clusters[merge_j])
            del clusters[merge_j]

    # Вычисляем центроиды (как средние координаты) для единообразия
    points = np.array(list(zip(lat, lon)))
    centroids = []
    for cl in clusters:
        if cl:
            cen = np.mean(points[cl], axis=0)
            centroids.append(cen)
        else:
            centroids.append([np.nan, np.nan])
    return clusters, np.array(centroids)
```

```

#2. K-means (ручная реализация)
def kmeans_manual(lat, lon, k, max_iter=100):
    n = len(lat)
    points = np.array(list(zip(lat, lon)))
    min_lat, max_lat = min(lat), max(lat)
    min_lon, max_lon = min(lon), max(lon)
    # случайная инициализация центроидов
    centroids = np.array([[random.uniform(min_lat, max_lat),
                           random.uniform(min_lon, max_lon)] for _ in
                           range(k)])

    for _ in range(max_iter):
        clusters = [[] for _ in range(k)]
        for i, p in enumerate(points):
            dists = [math.hypot(p[0]-c[0], p[1]-c[1]) for c in centroids]
            cl_idx = np.argmin(dists)
            clusters[cl_idx].append(i)
        new_centroids = []
        for c in range(k):
            if clusters[c]:
                avg_lat = np.mean([points[i][0] for i in clusters[c]])
                avg_lon = np.mean([points[i][1] for i in clusters[c]])
                new_centroids.append([avg_lat, avg_lon])
            else:
                # пустой кластер - случайная точка
                new_centroids.append([random.uniform(min_lat, max_lat),
                                       random.uniform(min_lon, max_lon)])
        new_centroids = np.array(new_centroids)
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids
    return clusters, centroids

```

```

#3. K-means из scikit-learn
def kmeans_sklearn(lat, lon, k):
    X = np.array(list(zip(lat, lon)))
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X)
    clusters = [[] for _ in range(k)]
    for i, lbl in enumerate(labels):
        clusters[lbl].append(i)
    return clusters, km.cluster_centers_

```

```

# Вспомогательные функции оценки
def inertia(clusters, lat, lon):
    """Сумма квадратов расстояний от точек до центров их кластеров."""
    points = np.array(list(zip(lat, lon)))
    total = 0.0
    for cl in clusters:
        if not cl:
            continue
        center = np.mean(points[cl], axis=0)

```

```

        for idx in cl:
            total += np.sum((points[idx] - center)**2)
    return total

def silhouette_score_own(labels, lat, lon):
    """Индекс силуэта (требуется метки для каждой точки)."""
    X = np.array(list(zip(lat, lon)))
    if len(set(labels)) < 2:
        return -1.0
    return silhouette_score(X, labels)

def time_algorithm(alg_func, lat, lon, k):
    """Замеряет время выполнения алгоритма и возвращает (clusters, время)."""
    start = time.perf_counter()
    clusters, _ = alg_func(lat, lon, k) # все функции возвращают (clusters,
    центроиды)
    elapsed = time.perf_counter() - start
    return clusters, elapsed

# Генерация тестовых конфигураций
def generate_test_configs():
    configs = []
    # 1. Три компактные группы
    lat1 = [10,10,10, 30,30,30, 50,50,50]
    lon1 = [10,11,12, 30,31,32, 50,51,52]
    configs.append(('3 компактные группы', lat1, lon1, 3))
    # 2. Случайные точки
    random.seed(42)
    lat2 = [random.uniform(0,100) for _ in range(15)]
    lon2 = [random.uniform(0,100) for _ in range(15)]
    configs.append(('Случайные точки', lat2, lon2, 3))
    # 3. Линейная структура
    lat3 = list(range(20))
    lon3 = [2*x for x in range(20)]
    configs.append(('Линейная структура', lat3, lon3, 2))
    # 4. Один выброс
    lat4 = [10,11,12,13,14, 100]
    lon4 = [10,11,12,13,14, 100]
    configs.append(('Один выброс', lat4, lon4, 2))
    # 5. Исходные города
    lat5 = [52.5, 32.1, 12.5, 87.5, 20.1]
    lon5 = [34.2, 12.1, 90.2, 80.1, 89.3]
    configs.append(('Исходные города', lat5, lon5, 2))
    return configs

#Запуск эксперимента
def run_experiment():
    configs = generate_test_configs()
    results = []
    for name, lat, lon, k in configs:
        print(f"\n=== {name} (K={k}) ===")

        # Агломеративный

```

```

cl_h, t_h = time_algorithm(hierarchical_clustering, lat, lon, k)
inert_h = inertia(cl_h, lat, lon)
labels_h = [0]*len(lat)
for ci, cl in enumerate(cl_h):
    for idx in cl:
        labels_h[idx] = ci
sil_h = silhouette_score_own(labels_h, lat, lon)

# K-means ручной
cl_m, t_m = time_algorithm(kmeans_manual, lat, lon, k)
inert_m = inertia(cl_m, lat, lon)
labels_m = [0]*len(lat)
for ci, cl in enumerate(cl_m):
    for idx in cl:
        labels_m[idx] = ci
sil_m = silhouette_score_own(labels_m, lat, lon)

# K-means sklearn
cl_s, t_s = time_algorithm(kmeans_sklearn, lat, lon, k)
inert_s = inertia(cl_s, lat, lon)
labels_s = [0]*len(lat)
for ci, cl in enumerate(cl_s):
    for idx in cl:
        labels_s[idx] = ci
sil_s = silhouette_score_own(labels_s, lat, lon)

results.append({
    'config': name, 'k': k,
    'hier_inertia': inert_h, 'hier_silhouette': sil_h, 'hier_time':
t_h,
    'manual_inertia': inert_m, 'manual_silhouette': sil_m,
'manual_time': t_m,
    'sklearn_inertia': inert_s, 'sklearn_silhouette': sil_s,
'sklearn_time': t_s
})

print(f"Агломеративный: inertia={inert_h:.2f},
silhouette={sil_h:.3f}, time={t_h:.5f}s")
print(f"K-means ручной: inertia={inert_m:.2f},
silhouette={sil_m:.3f}, time={t_m:.5f}s")
print(f"K-means sklearn: inertia={inert_s:.2f},
silhouette={sil_s:.3f}, time={t_s:.5f}s")

return results

if __name__ == "__main__":
    run_experiment()

```